Pattern Composite

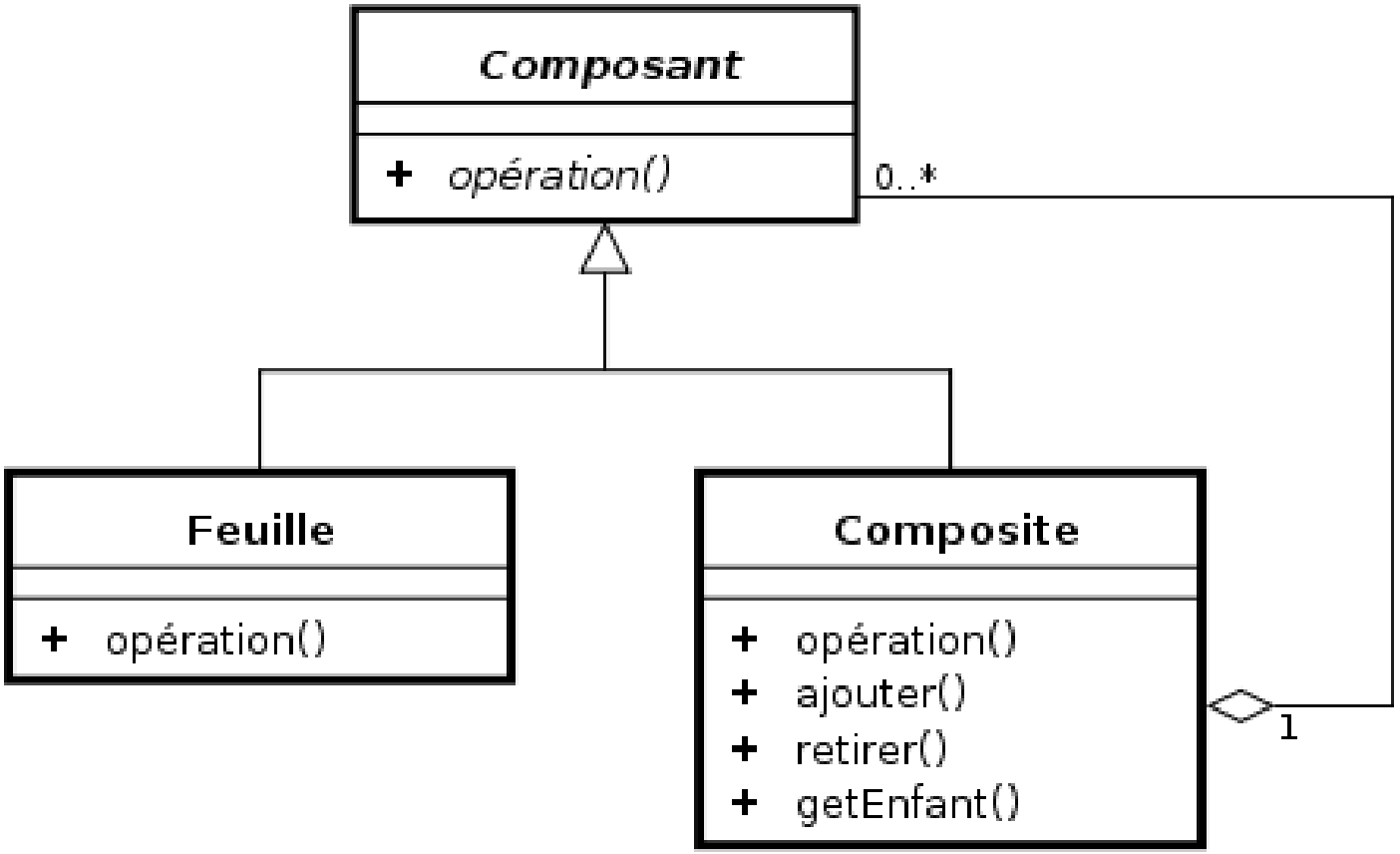
C'est quoi

- Le pattern **composite** est un patron de conception (*design pattern*) structurel. Ce patron permet de concevoir une structure d'arbre, comme un arbre binaire en limitant à 2 le nombre de sous-éléments.
- Un objet composite est constitué d'un ou de plusieurs objets similaires (ayant des fonctionnalités similaires). L'idée est de manipuler un groupe d'objets de la même façon que s'il s'agissait d'un seul objet. Les objets ainsi regroupés doivent posséder des opérations communes, c'est-à-dire un "dénominateur commun".

Quand l'utiliser

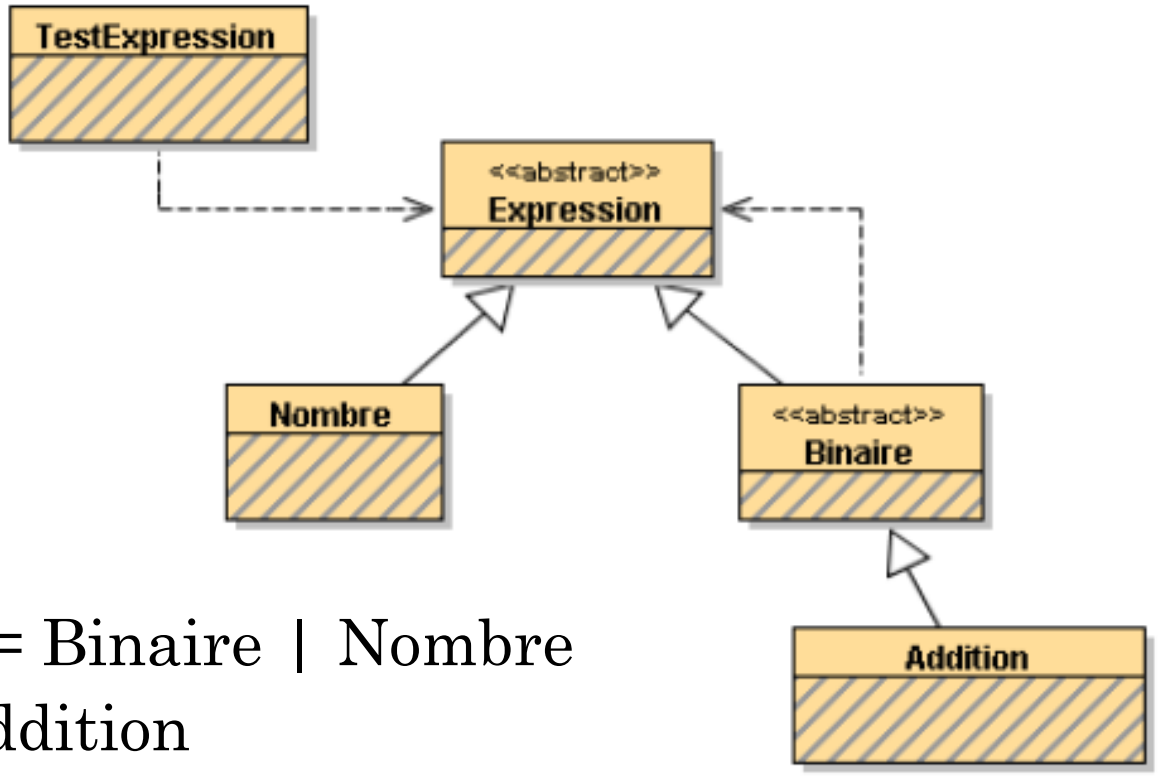
- Vous avez l'impression d'utiliser de multiples objets de la même façon, souvent avec des lignes de code identiques ou presque. Par exemple, lorsque la seule et unique différence entre deux méthodes est que l'une manipule un objet de type Carré, et l'autre un objet Cercle. Lorsque, pour le traitement considéré, la différenciation n'a *pas besoin* d'exister, il serait plus simple de considérer l'ensemble de ces objets comme homogène.

Diagramme de classe



- **Composant**
  - est l'abstraction pour tous les composants, y compris ceux qui sont composés
  - déclare l'interface pour le comportement par défaut
- **Feuille**
  - représente un composant n'ayant pas de sous-éléments
  - implémente le comportement par défaut
- **Composite**
  - représente un composant pouvant avoir des sous-éléments
  - stocke des composants enfants et permet d'y accéder
  - implémente un comportement en utilisant les enfants
- **Client**
  - manipule les objets de la composition à travers l'interface la classe Composant

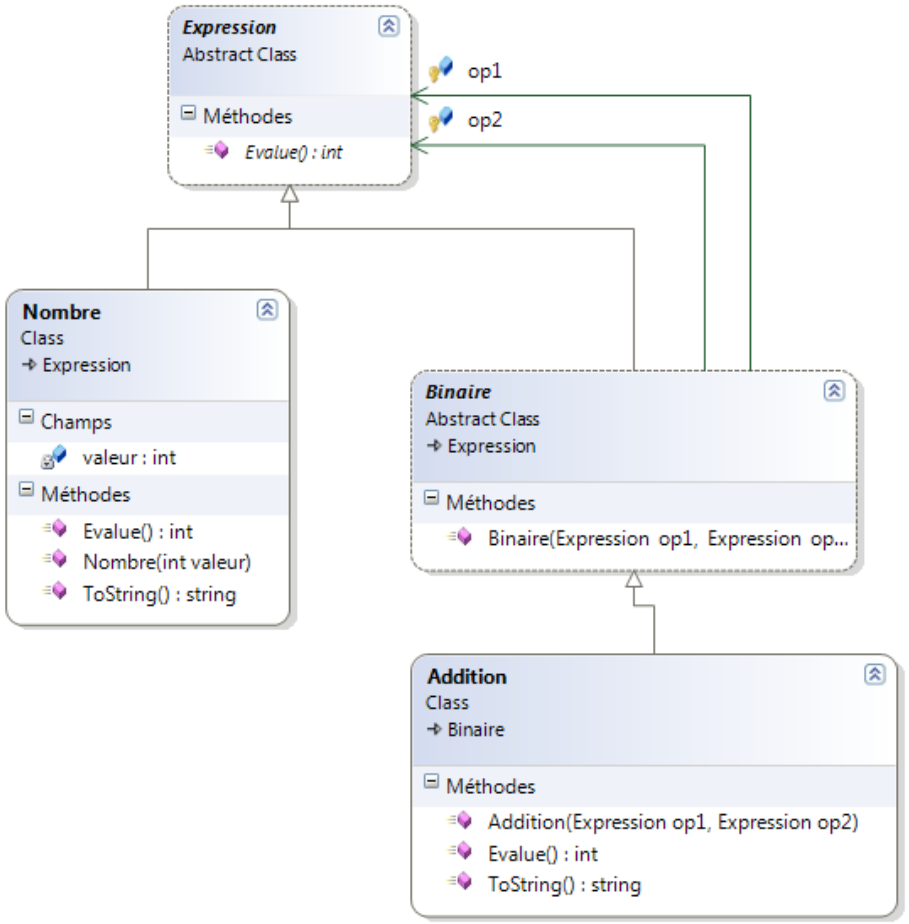
Exemple de composite:Expression 3 , 3+2, 3+2+5,…



- Expression ::= Binaire | Nombre
- Binaire ::= Addition
- Addition ::= Expression ‘+’ Expression
- Nombre ::= ‘une valeur de type int’
- *En bref ! Tout est Expression*

Exercice: Expression

- Nous vous fournissons le diagramme de classe permettant de réaliser des opérations d'addition entre entier.



Exercice: Expression

- Réalisez l'implémentation des différentes classes:
  - Binaire représente n'importe quelle opération entre deux expressions, et est également une expression.
  - Entier représente une entier(2,3,89) et est une expression.
  - Addition est un binaire particulier.
  - Evalue return évaluation correspondant à l'expression

Exercice: Expression

- Tester votre code avec les exemples suivants

```
ClassLibraryExpression.Expression exp1 = new Addition(new Entier(33),new Entier(33));
int resultat1 = exp1.Evalue();

ClassLibraryExpression.Expression exp2 = new Addition(new Entier(33),new Addition(new Entier(33),new Entier(11)));
int resultat2 = exp2.Evalue();
```

Exercice: Expression

- 1) Ajouter le binaire Soustraction
- 2) Améliorer votre diagramme afin d'avoir une méthode Formate qui renvoie un string pour représenter une expression:
  - Ex: 3+2=5, ou 3-6+7 = 4

Exercice: Expression

- Valider au niveau du formateur!

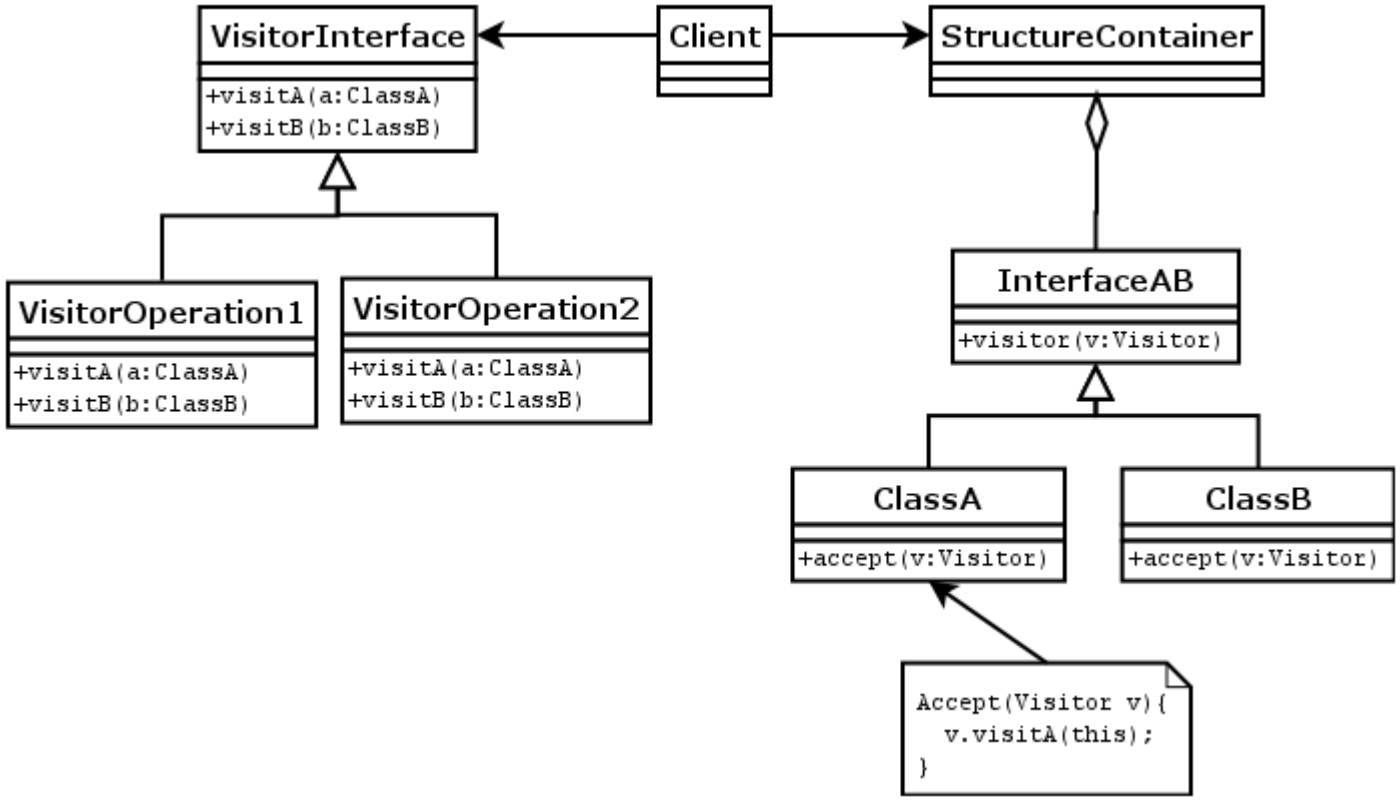
Exercice: Les figures géométriques

- Une figure géométrique peut être un carré, un triangle, un cercle, une ligne… ou un ensemble de figures.
- Proposez un diagramme de classe permettant de représenter toutes ces figures à votre formateur?
- Implémenter vos classes.

Le pattern visiteur

- Si les opérations sont identiques pour chaque classe, au lieu de dupliquer cette méthode, il est préférable de mettre ces opérations dans un *visiteur* (centralisation de l'opération).
- Les figures géométriques ont tous le comportement de se dessiner dans une *WinForm* **ou** de s'afficher dans une *Console*, suivant le projet exécutable que l'on réalise. On peut sortir ces comportements des classes et les centraliser dans d'autres classes afin de ne pas alourdir les classes du pattern composite.

Le pattern visiteur



Exercice: Le pattern visiteur dans l'exemple des figures géométriques

- L'interface **IVisiteurFigure** doit définir le contrat que doit respecter le future visiteur pour l'application WinForm et l'application Console.

```
public interface IVisiteur<T>
{
    T Visit(Cercle _c);
    T Visit(Rectangle _r);
    //etc..
    T Visit(Figures _r);
}
```

Exercice: Le pattern visiteur dans l'exemple des figures géométriques pour l'application console

- Implémentez cette interface, dans le projet **ConsoleApplicationFigures** fourni, la classe **VisiteurFigurePourConsole** qui réalise cette interface. La méthode Visit(Cercle _c) permet d'afficher dans la console les informations sur le cercle.
- Tester votre code.

Exercice: Le pattern visiteur dans l'exemple des figures géométriques pour la WinForm

- Implémentez cette interface, dans le projet **WindowsFormsPaint** fourni, la classe **VisiteurFigurePourWinForm** qui réalise cette interface. La méthode Visit(Cercle _c) permet de dessiner dans un graphics (une surface de dessin GDI) un cercle.
- Réalisez l'implémentation des menus contextuelles proposés